# Accelerating Secure Deep Learning with Fully Homomorphic Encryption

**Minwoo Yun**

GREAT Program Symposium

Biaslab · Professor. Mohsen Imani · Mentor. Sanggeon Yun

BACKGROUND

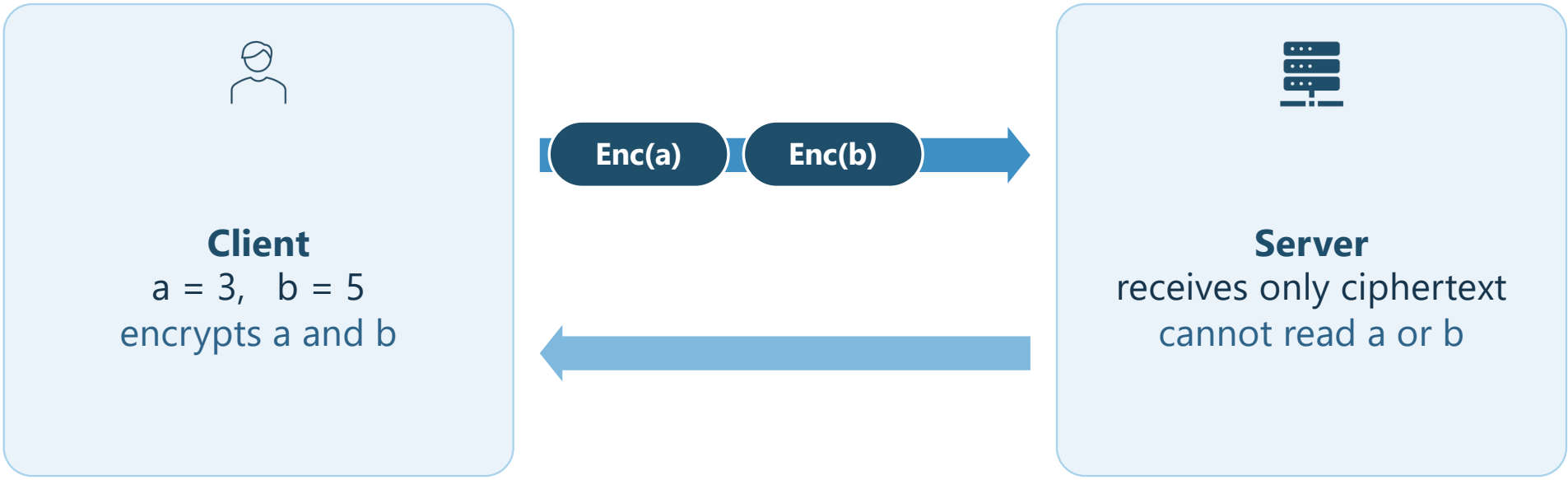
# What is Fully Homomorphic Encryption?

- Computation **directly on encrypted data**
- **No decryption** during computation
- **Same result** as plaintext computation
- Sensitive data **hidden from the server**

$$\mathbf{Dec( f( Enc(x) ) ) = f(x)}$$

EXAMPLE

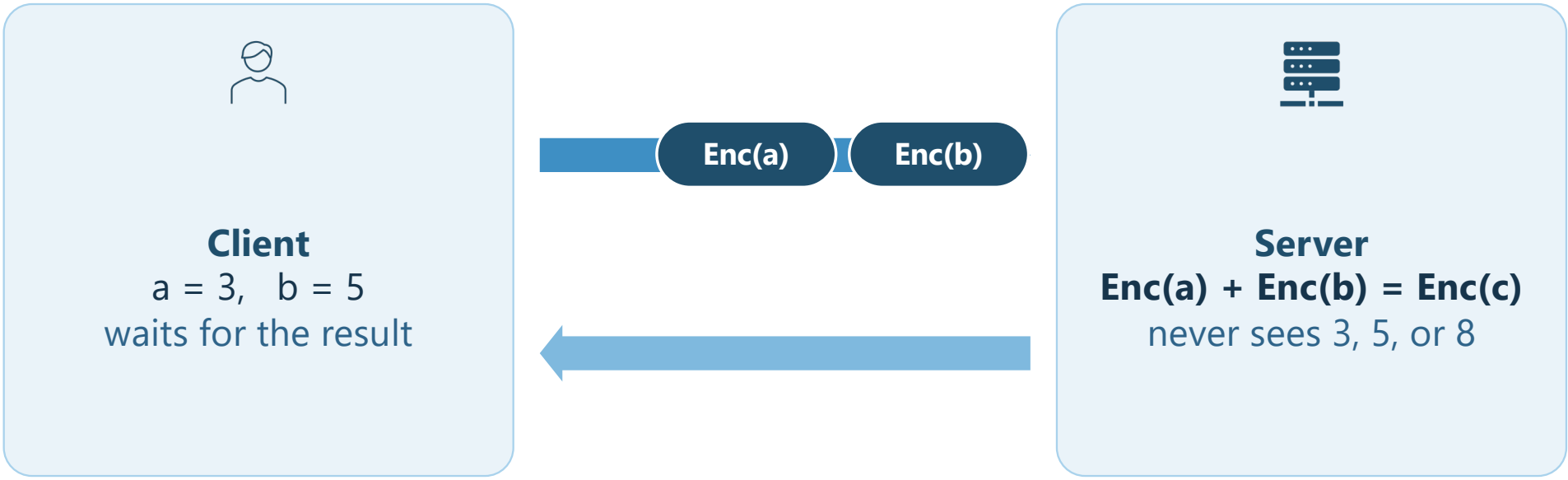
# Privacy Inference with FHE




**Only ciphertexts leave the client, so a and b stay private**

EXAMPLE

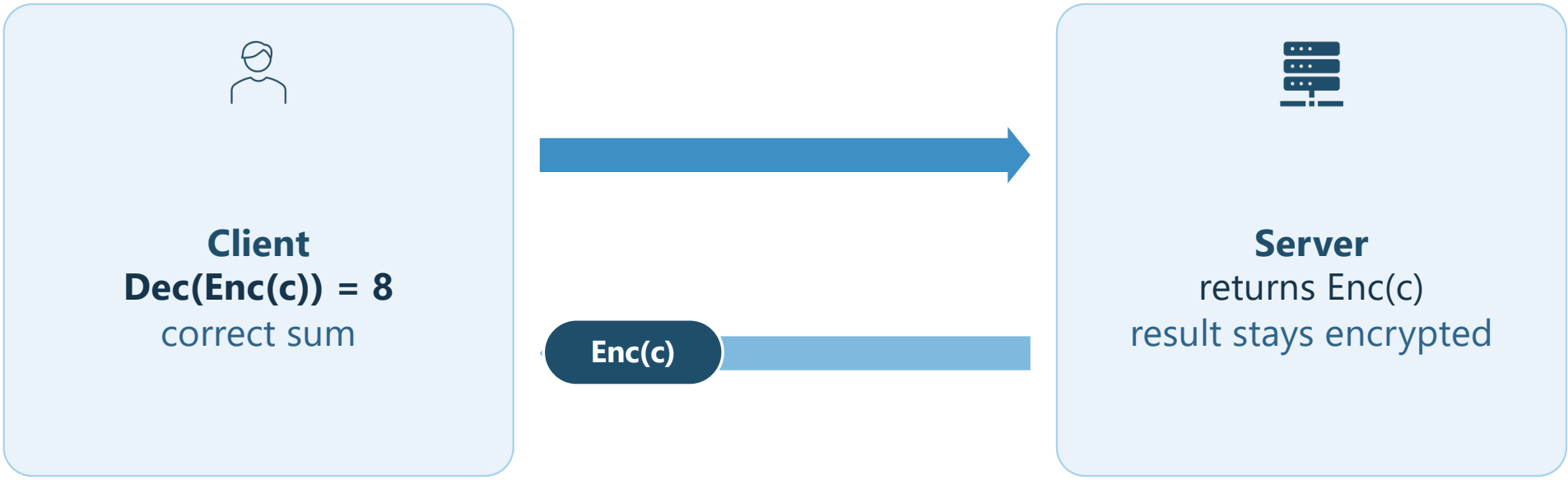
# Privacy Inference with FHE




**Homomorphic addition: the sum is computed without any decryption**

EXAMPLE

# Privacy Inference with FHE




**Only the client can decrypt, so the server never learns the answer**

PROBLEM

# Strong Privacy Comes at a Cost



**Private**
the server never sees the raw data



**But very slow**
far slower than plaintext inference



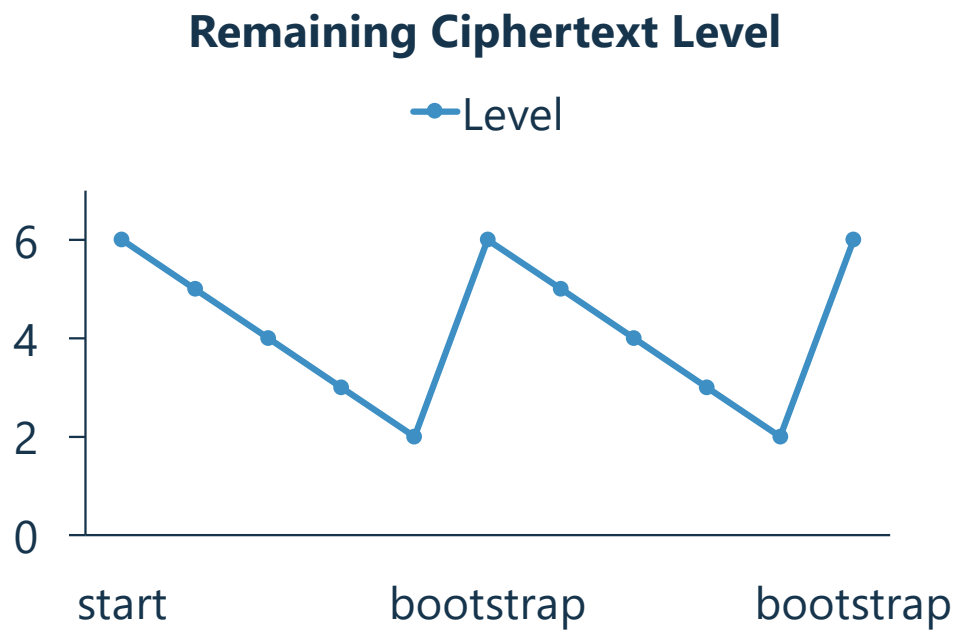
**Bootstrapping**
the dominant, expensive operation

**The number of bootstrappings is tied to the multiplicative depth of the network**

PROBLEM

# Why Is FHE Inference So Slow?

- Main bottleneck: **bootstrapping**
- Each multiplication **consumes levels**
- No level left: **bootstrapping refreshes**
- Very expensive, **much longer inference**

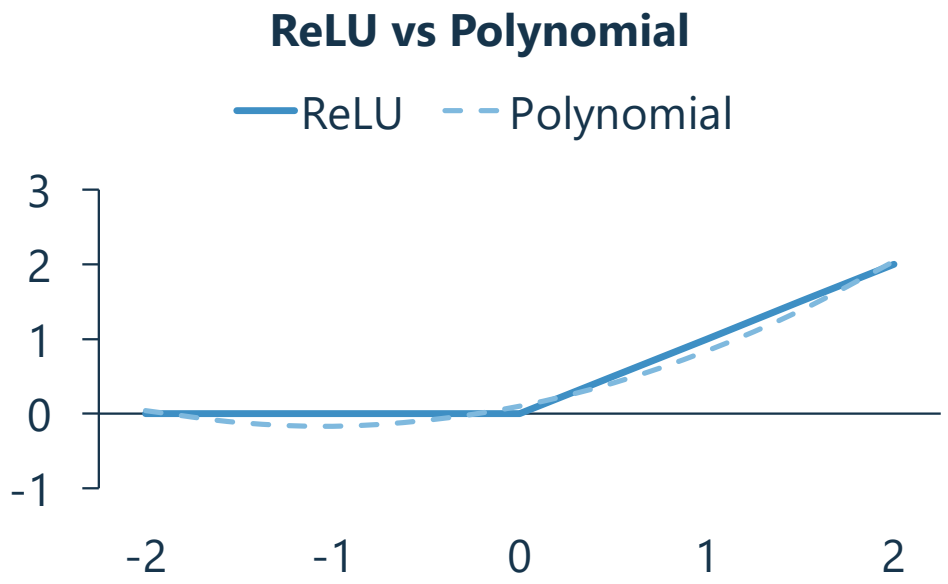



**So what consumes the levels? Multiplications**

PROBLEM

# Where Do the Multiplications Come From?

- **CKKS**[1, 2]: additions and multiplications only
- ReLU: **not a polynomial**
- Needs **polynomial approximation**
- **More multiplications**, deeper network




**Polynomial approximation is the main source of extra multiplications**

[1] *Cheon et al. Homomorphic encryption for arithmetic of approximate numbers. ASIACRYPT 2017.*
[2] *Al Badawi et al. OpenFHE: open source fully homomorphic encryption library. WAHC 2022.*

PRIOR WORK

# Making Each Bootstrapping Faster



**Accelerate each bootstrapping operation**

moving the computation from CPUs to GPUs

**But the number of bootstrappings stays the same, so inference is still slow**

RESEARCH QUESTION

# Can We Cut Multiplications and Keep Performance?

**Shrinking the model cuts multiplications easily**

but accuracy collapses along with it

**More multiplications** → **More level consumption** → **More bootstrapping**

**Our answer: the memory centric architecture**

APPROACH

# Cutting Multiplications: Network Folding




consecutive teacher blocks

**Knowledge distillation folds them into one student block: one block = one bootstrapping**

and we choose how far to fold, that is, the model size

**Wider blocks for free**
use spare ciphertext slots

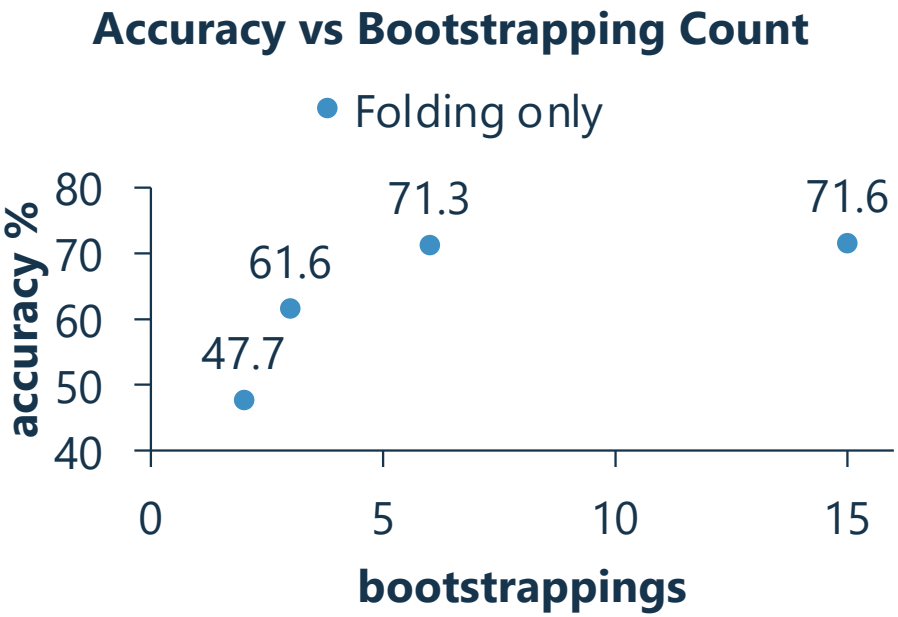
**Learned low degree ReLU**
halves level consumption

**Sparser bootstrapping**
once per two blocks, 12 to 21% faster

RESULT

# Folding Alone Hits a Performance Cliff

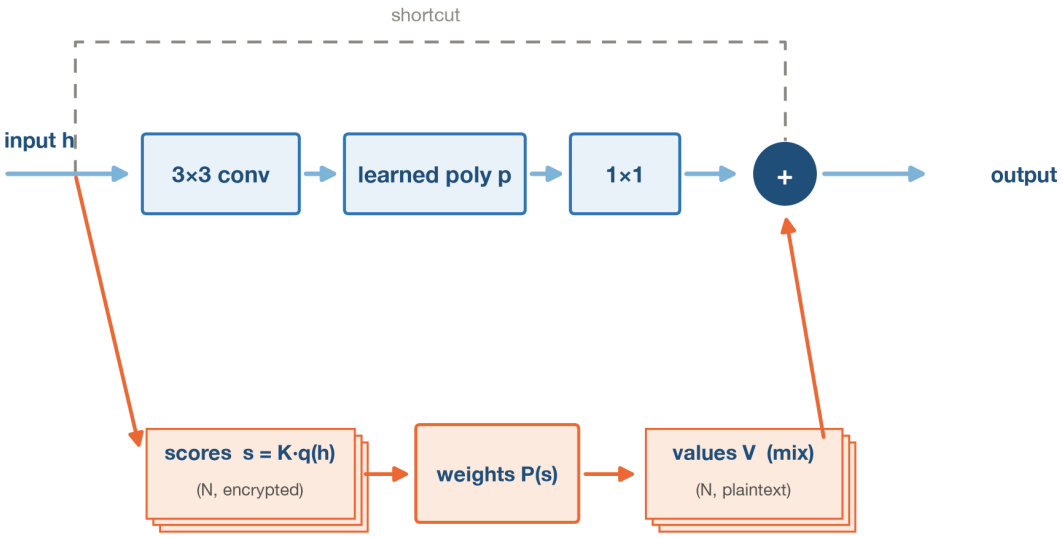
- CIFAR 100, ResNet 32, all numbers measured on ciphertext
- deeper folding, **fewer bootstrappings, faster inference**
- at 6 blocks: **teacher level accuracy, 12× faster**
- **further folding: a cliff in accuracy**




**Multiplications went down, but performance fell with them**

ANSWER

# The Memory Centric Architecture[1]




memory path: parallel to the block, joined at the add, depth = max of the two paths

- the backbone stays unchanged
- the memory path runs in parallel
- it gives back the capacity lost by folding
- **it adds zero bootstrappings**

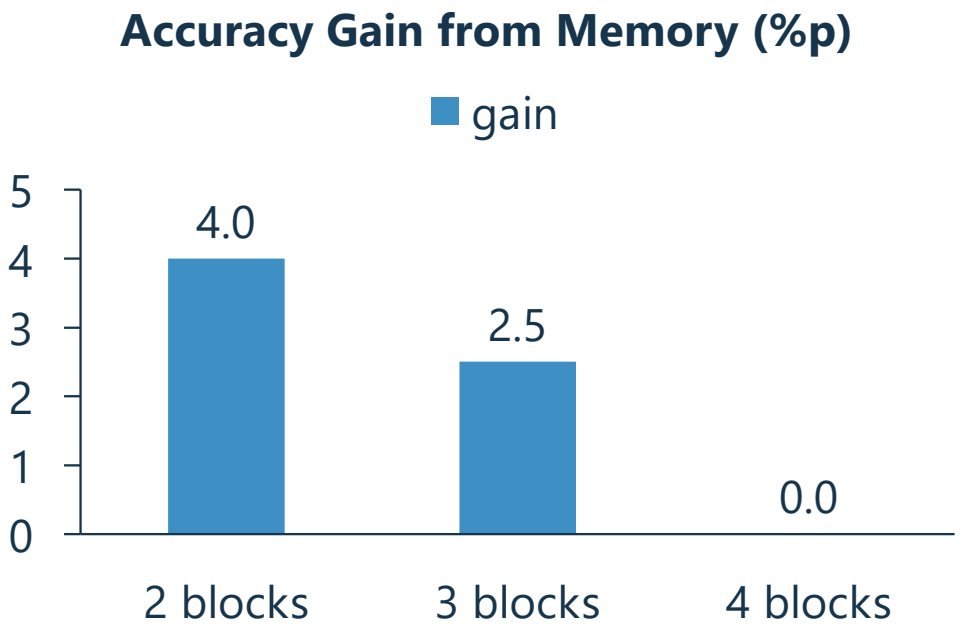
**On the multiplication budget, memory capacity is free**

[1] *Lample et al. Large memory layers with product keys. NeurIPS 2019.*

RESULT

# Shrink the Model, Keep the Performance

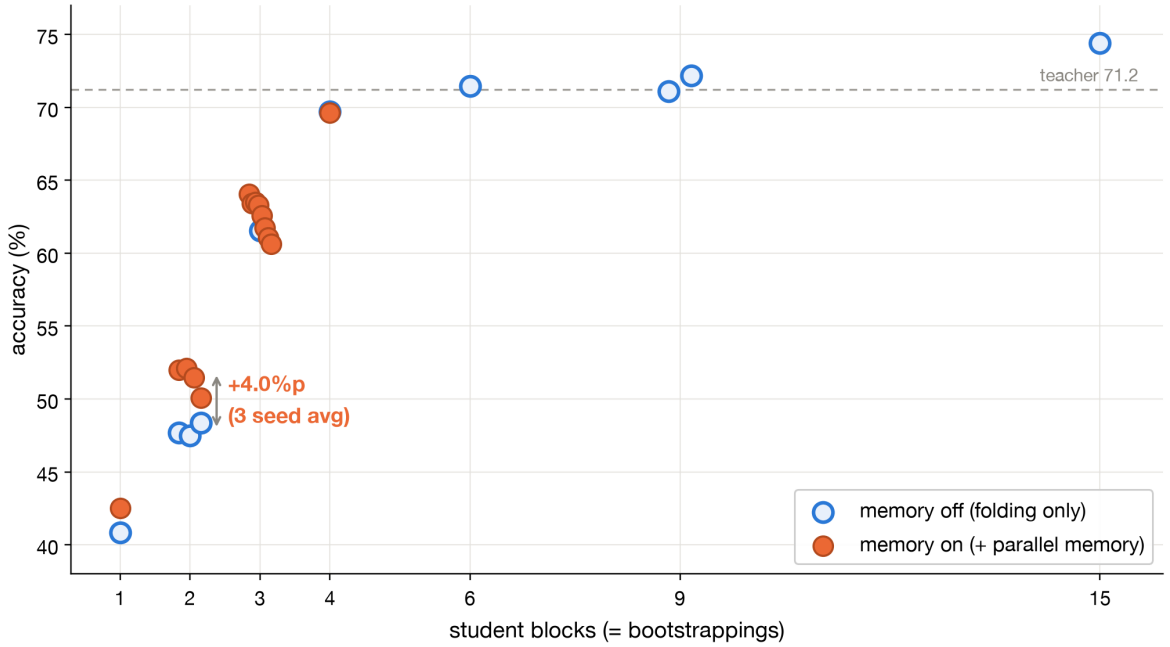
- same bootstrapping budget, memory on vs off
- smaller model, **larger memory contribution**
- peak gain at 2 blocks, zero once the backbone suffices
- growing reliance on memory as compression deepens




**The smaller the model, the more memory gives back**

SUMMARY

# Folding and Memory: The Full Picture




**Keep the architecture, shrink the model, and memory keeps the performance**

SUMMARY

# One Question, One Tool, One Answer

**QUESTION** — **Can we cut multiplications and keep performance scaling?**

**FOLDING** — Cut the number of bootstrappings itself: **teacher level accuracy, 12x faster**, fully measured on ciphertext

**MEMORY** — Zero added bootstrappings, **+4.0%p back on average**, and the smaller the model, the bigger the gain

FUTURE WORK

# What Comes Next

**SCALE** memory at ImageNet scale: **larger task, larger gain?**

**COST** lower time cost via **pruning entries unused across all inputs** (usage 30 to 62%)

# Thank You
## Questions & Discussion

**Minwoo Yun**
GREAT Program Symposium

Biaslab · Professor. Mohsen Imani · Mentor. Sanggeon Yun